

# Python Exercise — Graph Representation of a Road Network

Jiwon Kim, The University of Queensland

2026

## Contents

|                                                                 |          |
|-----------------------------------------------------------------|----------|
| <b>Python Exercise — Graph Representation of a Road Network</b> | <b>2</b> |
| 1. Background . . . . .                                         | 2        |
| 2. The Network . . . . .                                        | 2        |
| 3. Task 1 — Create the Link DataFrame . . . . .                 | 3        |
| 4. Task 2 — Identify the Set of Nodes . . . . .                 | 3        |
| 5. Task 3 — Build a Downstream Nodes Dictionary . . . . .       | 4        |
| 6. Reflection Questions . . . . .                               | 4        |

# Python Exercise — Graph Representation of a Road Network

## 1. Background

In transport network modelling, a road network is represented as a **directed graph**  $G = (N, A)$ , where:

- $N$  is the set of **nodes** (intersections or zone centroids)
- $A$  is the set of **directed links** (road segments), each connecting an upstream node to a downstream node

Each link  $(i, j) \in A$  has attributes such as **length**, **capacity**, and **free-flow travel time**.

## 2. The Network

Consider the following small road network with **5 nodes** and **7 directed links**:

| Link  | From node | To node | Length (km) |
|-------|-----------|---------|-------------|
| (1,2) | 1         | 2       | 3.0         |
| (1,3) | 1         | 3       | 5.0         |
| (2,3) | 2         | 3       | 2.0         |
| (2,4) | 2         | 4       | 4.0         |
| (3,4) | 3         | 4       | 1.0         |
| (3,5) | 3         | 5       | 6.0         |
| (4,5) | 4         | 5       | 2.0         |

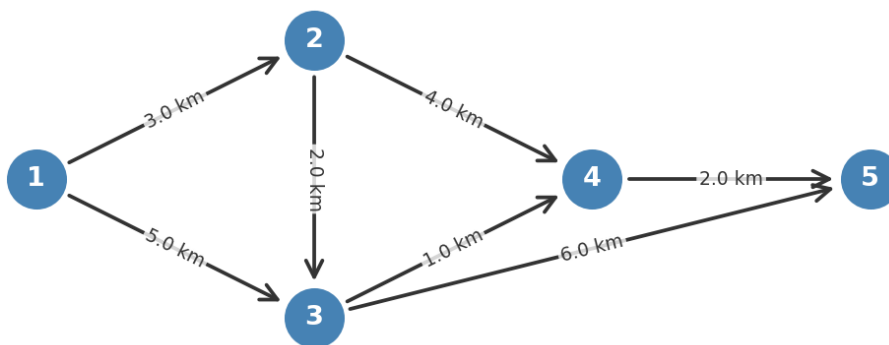


Figure 1: Road network

### 3. Task 1 — Create the Link DataFrame

Create a **dictionary** called `link_data` with three keys: `'from_node'`, `'to_node'`, and `'length'`. Each key maps to a list of values, one entry per link (in the order shown in the table above).

Then use `pd.DataFrame()` to convert it into a DataFrame called `df_links`. Print the DataFrame.

```
import pandas as pd

# --- Your code below ---

link_data = {
    'from_node': [...], # fill in
    'to_node': [...], # fill in
    'length': [...], # fill in
}

df_links = ...

print(df_links)
```

Expected output:

|   | from_node | to_node | length |
|---|-----------|---------|--------|
| 0 | 1         | 2       | 3.0    |
| 1 | 1         | 3       | 5.0    |
| 2 | 2         | 3       | 2.0    |
| 3 | 2         | 4       | 4.0    |
| 4 | 3         | 4       | 1.0    |
| 5 | 3         | 5       | 6.0    |
| 6 | 4         | 5       | 2.0    |

---

### 4. Task 2 — Identify the Set of Nodes

Extract all node IDs that appear in `df_links` (in both the `'from_node'` and `'to_node'` columns), remove duplicates, and store them in a **sorted list** called `nodes`. Print the result.

Expected output:

```
[1, 2, 3, 4, 5]
```

*There are multiple valid ways to do this — think about what tools you have available:*

- Python's built-in `set()` removes duplicates from any list; `sorted()` then returns a sorted list.
- Pandas columns have a `.unique()` method that returns unique values, and `pd.concat()` can combine two columns into one Series before calling it.
- A set union (`|`) can also merge the unique values from two columns directly.

*Try one approach first, then see if you can find an alternative.*

**Tip — always check the official documentation.** Rather than asking for a single example, look up how these functions work in general — you will find more options and edge cases than any single example can show:

- [set\(\)](#) — Python docs
  - [sorted\(\)](#) — Python docs
  - [Series.unique\(\)](#) — pandas docs
  - [pd.concat\(\)](#) — pandas docs
- 

## 5. Task 3 — Build a Downstream Nodes Dictionary

Create a dictionary called `downstream_nodes` where:

- Each **key** is a node ID (from `nodes`)
- Each **value** is a **list of directly reachable downstream nodes**

For example, node 1 connects to nodes 2 and 3, so `downstream_nodes[1]` should be `[2, 3]`.

Print the result.

**Expected output:**

```
{1: [2, 3], 2: [3, 4], 3: [4, 5], 4: [5], 5: []}
```

---

## 6. Reflection Questions

1. Node 5 has no outgoing links. What does this mean in terms of the road network?
2. If you were to find the shortest path from node 1 to node 5, which nodes would you need to visit first?
3. How could the `downstream_nodes` dictionary help you systematically explore the network?